



TRADECHAIN

Secure Blockchain-Based Trading Ledger System

ABSTRACT

A Python-based blockchain trading simulation demonstrating secure, transparent, and tamper-proof digital transactions through hashing and Proof of Work.

Contents

1. INTRODUCTION	2
2. OBJECTIVES OF THE PROJECT	2
3. REAL-LIFE APPLICATION OF BLOCKCHAIN	2
Applications include:	2
Working in Real Life:	2
4. SYSTEM DESIGN / ARCHITECTURE	3
4.1 Block Structure	3
4.2 Blockchain Structure.....	3
4.3 Trading System	3
4.4 Menu System	3
5. IMPLEMENTATION DETAILS.....	3
Technologies Used:	3
Key Concepts Implemented:.....	4
6. FEATURES OF THE SYSTEM	4
7. ADVANTAGES	4
9. TESTING AND RESULTS.....	5
Test Case 1:.....	5
Test Case 2:.....	5
Test Case 3:.....	5
Test Case 4:.....	5
10. FUTURE ENHANCEMENTS	5
11. CONCLUSION	5
APPENDIX.....	5
Source Code:	5

1. INTRODUCTION

Blockchain technology is a decentralized and distributed digital ledger system that records transactions securely across multiple nodes. It ensures transparency, security, and immutability of data.

The TradeChain project is designed to simulate a basic blockchain-based trading system where users can perform transactions, and each transaction is securely stored in a blockchain. The system demonstrates how blockchain prevents tampering and ensures trust in digital systems without requiring a central authority.

2. OBJECTIVES OF THE PROJECT

The main objectives of this project are:

- To understand the working of blockchain technology
 - To implement a simplified blockchain system in Python
 - To simulate secure digital transactions between users
 - To demonstrate hashing and Proof of Work concepts
 - To ensure data integrity and immutability
 - To provide a basic trading system using blockchain
-

3. REAL-LIFE APPLICATION OF BLOCKCHAIN

Blockchain is widely used in modern digital systems where security and transparency are required.

Applications include:

- **Cryptocurrency Systems:** Bitcoin, Ethereum use blockchain for secure transactions
- **Banking Systems:** Secure and verified digital payments
- **Supply Chain Management:** Tracking goods from origin to delivery
- **Healthcare Systems:** Secure storage of patient records
- **Financial Trading Platforms:** Transparent and tamper-proof transaction records

Working in Real Life:

- Transactions are grouped into blocks
- Each block is linked using cryptographic hashes
- Any modification breaks the chain
- All participants maintain a synchronized ledger

TradeChain simulates this real-world mechanism at a basic level.

4. SYSTEM DESIGN / ARCHITECTURE

The TradeChain system consists of the following components:

4.1 Block Structure

Each block contains:

- Index (block position)
 - Timestamp (creation time)
 - Transaction data
 - Previous block hash
 - Current block hash
 - Nonce (used in mining)
-

4.2 Blockchain Structure

- A sequential chain of blocks
 - Each block depends on the previous block
 - Ensures immutability and integrity
-

4.3 Trading System

- Users have wallets with balances
 - Users can send and receive coins
 - Each transaction is stored in blockchain
-

4.4 Menu System

- Interactive system for user input
 - Allows creation of users and transactions
 - Displays blockchain and wallet data
-

5. IMPLEMENTATION DETAILS

The project is implemented using Python programming language.

Technologies Used:

- Python 3
- hashlib (SHA-256 hashing)

- time module (timestamps)

Key Concepts Implemented:

- Object-Oriented Programming (OOP)
 - SHA-256 Cryptographic Hashing
 - Proof of Work (Mining mechanism)
 - Blockchain linking using previous hash
 - Transaction-based ledger system
-

6. FEATURES OF THE SYSTEM

- User wallet creation system
 - Secure transaction processing
 - Blockchain-based storage of transactions
 - Proof of Work mining mechanism
 - Blockchain validation system
 - Interactive menu-driven interface
-

7. ADVANTAGES

- Provides high data security
 - Prevents tampering of transaction records
 - Ensures transparency in transactions
 - Eliminates need for central authority
 - Demonstrates real-world blockchain behavior
-

8. LIMITATIONS

- High computational cost due to mining process
 - Slow processing compared to traditional databases
 - Not suitable for large-scale deployment
 - Simulation does not include real network decentralization
 - Requires manual execution
-

9. TESTING AND RESULTS

Test Case 1:

Users created: Alice (100), Bob (50)

Test Case 2:

Transaction: Alice sends 20 to Bob

Result: Transaction successfully stored in blockchain

Test Case 3:

Blockchain validation executed

Result: Chain verified successfully

Test Case 4:

If any block is modified manually

Result: Blockchain validation fails (corruption detected)

10. FUTURE ENHANCEMENTS

- Integration of Smart Contracts
 - Graphical User Interface (GUI)
 - Real-time peer-to-peer network simulation
 - Fraud detection system
 - AI-based anomaly detection
 - Web-based blockchain dashboard
-

11. CONCLUSION

The TradeChain project successfully demonstrates the core principles of blockchain technology including hashing, Proof of Work, and immutable ledger structure.

It provides a simplified but effective simulation of secure digital trading systems used in real-world applications such as cryptocurrencies and financial platforms. The project enhances understanding of blockchain security and decentralized systems.

APPENDIX

Source Code:

```
import hashlib
```

```

import time

# BLOCK CLASS

class Block:
    def __init__(self, index, data, previous_hash, difficulty=2):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.proof_of_work(difficulty)

    def compute_hash(self):
        block_string =
f"{self.index}{self.timestamp}{self.data}{self.previous_hash}{self.nonce}"
        return hashlib.sha256(block_string.encode()).hexdigest()

    def proof_of_work(self, difficulty):
        prefix = "0" * difficulty

        while True:
            hash_value = self.compute_hash()
            if hash_value.startswith(prefix):
                return hash_value
            self.nonce += 1

# BLOCKCHAIN CLASS

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")

    def add_block(self, data, difficulty=2):
        last_block = self.chain[-1]
        new_block = Block(len(self.chain), data, last_block.hash, difficulty)
        self.chain.append(new_block)

```

```

def is_valid(self):
    for i in range(1, len(self.chain)):
        current = self.chain[i]
        previous = self.chain[i - 1]

        if current.hash != current.compute_hash():
            return False

        if current.previous_hash != previous.hash:
            return False

    return True

def display(self):
    for block in self.chain:
        print("\n-----")
        print(f"Index: {block.index}")
        print(f>Data: {block.data}")
        print(f"Hash: {block.hash}")
        print(f"Prev Hash: {block.previous_hash}")

# TRADE SYSTEM

class TradeSystem:
    def __init__(self, blockchain):
        self.blockchain = blockchain
        self.wallets = {}

    # Add user
    def add_user(self):
        name = input("Enter user name: ")
        balance = int(input("Enter initial balance: "))
        self.wallets[name] = balance
        print(f"User {name} added successfully!")

    # Show wallets
    def show_wallets(self):
        print("\n--- Wallets ---")
        for user, bal in self.wallets.items():
            print(f"{user}: {bal}")

    # Create transaction
    def create_transaction(self):

```



```

        sender = input("Sender: ")
        receiver = input("Receiver: ")
        amount = int(input("Amount: "))

        if sender not in self.wallets or receiver not in self.wallets:
            print("User not found!")
            return

        if self.wallets[sender] < amount:
            print("Insufficient balance!")
            return

        # update balances
        self.wallets[sender] -= amount
        self.wallets[receiver] += amount

        transaction = {
            "from": sender,
            "to": receiver,
            "amount": amount
        }

        self.blockchain.add_block(transaction)
        print("Transaction added to blockchain!")

    # validate chain
    def validate_chain(self):
        if self.blockchain.is_valid():
            print("Blockchain is valid")
        else:
            print("Blockchain is corrupted")

# MENU SYSTEM

def menu():
    chain = Blockchain()
    system = TradeSystem(chain)

    while True:
        print("\n===== TRADECHAIN MENU =====")
        print("1. Add User")
        print("2. Show Wallets")
        print("3. Create Transaction")

```

```

print("4. View Blockchain")
print("5. Validate Blockchain")
print("6. Exit")

choice = input("Enter choice: ")

if choice == "1":
    system.add_user()

elif choice == "2":
    system.show_wallets()

elif choice == "3":
    system.create_transaction()

elif choice == "4":
    chain.display()

elif choice == "5":
    system.validate_chain()

elif choice == "6":
    print("Exiting TradeChain...")
    break

else:
    print("Invalid choice!")

# RUN PROGRAM

if __name__ == "__main__":
    menu()

```